

Tarefa 3: Contador com Processos e Threads

19 de agosto de 2026

Resultados observados

No `cont1.c`, em todas as 3 execuções, cada processo filho imprimiu 1000000, enquanto o processo pai imprimiu 0.

No `cont2.c`, o valor teórico esperado seria 4000000, mas os resultados reais foram sempre menores e variaram entre execuções.

Execução	Valor final em <code>cont2.c</code>
1	1431347
2	1305740
3	1171730
4	1306585
5	1235595
6	1292238
7	1222249
8	1681956
9	1240474
10	1347679

Discussão

No `cont1.c`, o comportamento ocorre porque `fork()` cria processos com espaços de memória independentes. Cada filho recebe uma cópia da variável `contador`, faz seus próprios incrementos e imprime 1000000. O processo pai não compartilha essa memória e, por isso, permanece com 0.

No `cont2.c`, todas as threads compartilham a mesma variável global `contador`. Entretanto, a operação `contador++` não é atômica. Assim, múltiplas threads podem ler e escrever o contador ao mesmo tempo, causando perda de atualizações. Isso caracteriza uma condição de corrida e explica os valores finais incorretos.

Problemas e soluções

No `cont1.c`, o problema é a ausência de compartilhamento real do contador entre os processos. A solução adequada é usar memória compartilhada ou outro mecanismo de comunicação entre processos.

No `cont2.c`, o problema é a falta de sincronização no acesso à variável compartilhada. A solução adequada é proteger a região crítica com `mutex` ou usar operações atômicas.

Conclusão

Os resultados mostram que processos e threads possuem comportamentos distintos em relação à memória. Em `cont1.c`, os incrementos não chegam ao processo pai porque cada processo trabalha com sua própria cópia da variável. Em `cont2.c`, a memória é compartilhada, mas a ausência de sincronização produz resultados incorretos.